

TP01 — Installer ShopLab et observer une requête

Séances : S01–S02 · Durée : 180 min · Compétences : C01, C02 · Niveau : guidé

Contrat de sécurité

Travaillez uniquement sur `http://127.0.0.1` et `https://127.0.0.1`. Arrêtez-vous si une commande contient une autre cible. Les comptes `alice`, `bob` et `admin` et toutes les données du lab sont synthétiques. Ne joignez jamais de token complet au compte rendu.

Objectifs et prérequis

À la fin, vous saurez vérifier l'hôte, démarrer et arrêter le lab, suivre DNS/TCP/TLS/HTTP/proxy/API/DB et produire une preuve expurgée. Prérequis : terminal, Git, Docker avec Compose v2, `curl`, OpenSSL et Python 3. Depuis la racine du dépôt, utilisez `./scripts/verify-host.sh` avant de commencer.

Livrables et critères de réussite

- un tableau `étape → observation → preuve → conclusion` ;
- une trace HTTP annotée, un certificat décrit sans clé privée et un schéma du flux ;
- `verify-lab.sh` terminé par `FAIL=0`, puis preuve de cleanup sans conteneur restant.

Mise en place (20 min)

```
cd labs/shoplab
../../scripts/verify-host.sh
./scripts/lab-start.sh core
./scripts/health-check.sh core
```

Résultat attendu : des lignes `[PASS]`, l'URL locale et `mode=vulnerable`. Un `[WARN]` demande une vérification manuelle; un `[FAIL]` bloque la suite.

A — Scope et dossier de preuve (20 min)

Créez un dossier hors Git `preuves/TP01/`. Dans `scope.md`, notez cible autorisée, heure de début, outils, stop conditions et règle d'expurgation. Relevez `git rev-parse --short HEAD`, `docker version` et `docker compose version`.

B — Requête HTTP (35 min)

```
curl -sv -H 'X-Correlation-ID: tp01-groupe-XX' \
  http://127.0.0.1:${SHOPLAB_HTTP_PORT:-8080}/api/health \
  -o preuves/TP01/health.json 2>preuves/TP01/http-trace.txt
```

Annotez méthode, chemin, `Host`, statut, type de contenu et identifiant de corrélation. Expliquez pourquoi un statut HTTP n'est pas la preuve que toute la chaîne est saine.

C — TCP et TLS (35 min)

```
openssl s_client -connect 127.0.0.1:${SHOPLAB_HTTPS_PORT:-8443} \
  -servername localhost </dev/null 2>preuves/TP01/tls.txt
curl -skI https://127.0.0.1:${SHOPLAB_HTTPS_PORT:-8443}/
```

Relevez sujet, émetteur, SAN, dates, version TLS et cipher. Le certificat est auto-signé : `-k` est toléré uniquement ici. Expliquez la différence entre chiffrement du transport et confiance dans l'identité.

D — De proxy à la base (45 min)

```
docker compose --profile core ps
docker compose --profile core logs --tail=30 proxy api db
curl -sS http://127.0.0.1:${SHOPLAB_HTTP_PORT:-8080}/api/products?q=lamp
```

Reliez chaque service au flux. Retrouvez votre `X-Correlation-ID` dans les logs API. Distinguez ce que vous avez observé de ce que vous inférez. Dessinez les frontières hôte/edge/core/base.

Vérification, reset et cleanup (20 min)

```
./scripts/verify-lab.sh basic
./scripts/reset.sh
./scripts/health-check.sh core
./scripts/cleanup.sh
docker compose --profile core ps --all
```

Le dernier tableau ne doit contenir aucun conteneur actif du projet. Le trap de test nettoie aussi après échec.

Dépannage

- port occupé : définissez `SHOPLAB_HTTP_PORT=18080 SHOPLAB_HTTPS_PORT=18443` ;
- daemon absent : Docker Desktop/Colima doit être démarré; sous Linux, vérifiez `docker info` ;
- certificat refusé : n'importez pas la CA; utilisez `-k` pour ce lab seulement ;
- WSL2 : placez le dépôt dans le système de fichiers Linux pour de meilleures performances.

Barème formatif /20

Critère	Insuffisant	Conforme	Maîtrisé	Points
Scope/éthique	cible non bornée	localhost et stop conditions	justification/expurgation	4
Chaîne technique	liste sans lien	flux complet correct	frontières et limites	6
Preuves	absentes/brutes	horodatées et expurgées	reproductibles et indexées	5
Cycle de vie	résidus	start/verify/reset/cleanup	diagnostic autonome	5